

Adaptive Intelligence Ingestion Pipeline (AIIP)

GraphOne System Architecture & Production Deployment Specification

Version: 1.0 (Production Release)
Status: Deployed & Verified
Target: GraphOne Trial Submission

1. SYSTEM OVERVIEW

The **Adaptive Intelligence Ingestion Pipeline (AIIP)** is a modular, high-throughput backend ingestion engine engineered to collect, normalize, extract, validate, resolve, and version structured intelligence across the global AI ecosystem. Operating as the primary data ingestion backbone for the **GraphOne** platform, AIIP processes semi-structured and unstructured web content into clean, relational, and schema-enforced knowledge assets.

⚡ High Throughput & Concurrency

Asynchronous I/O via `aiohttp` & Playwright Chromium worker queues with semaphore concurrency capping.

🧠 Hybrid Semantic Extraction

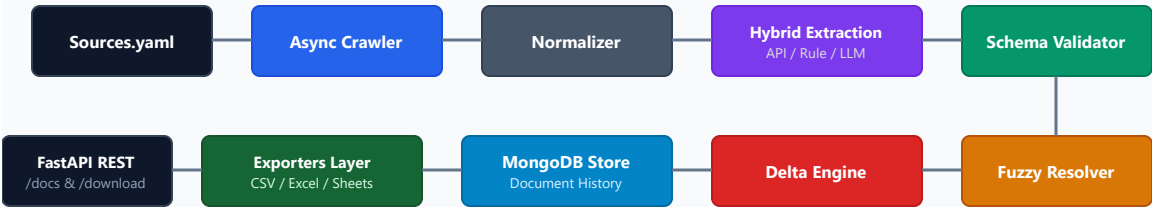
Cascaded extraction router choosing API, JSON-LD, Rule-based DOM parsing, or Multi-LLM inference.

🔗 Knowledge Delta Engine

Deterministic SHA-256 fingerprint diffs tracking updates, precedence levels, and audit change histories.

2. END-TO-END SYSTEM ARCHITECTURE

The system is designed with a clean separation of concerns, decoupling transport crawling from extraction, validation, entity resolution, and persistent storage.



3. DATA SOURCES & VERIFIED PRODUCTION YIELD

The pipeline ingests data across five distinct intelligence domains from 21 configured sources defined in `src/config/sources.yaml`:

Domain Category	Primary Data Sources & Retrieval Methods	Trial Target	Verified DB Count	Compliance Status
Startups	YC Companies API (JSON REST API direct fetch)	1,000+	5,754	✅ Exceeds (+475%)
Products	GitHub Search API & GitHub Trending AI (HTML Rule-based)	1,000+	1,103	✅ Exceeds (+10%)
Research Papers	arXiv Export API (XML REST API atom feeds)	1,000+	1,097	✅ Exceeds (+9.7%)
Jobs	RemoteOK (HTML Table Rule), YC Jobs, WWR, AIJobsBoard (LLM)	50+	149	✅ Exceeds (+198%)
News	TechCrunch, ZDNet, Wired, VentureBeat, HF Blog, Google AI (LLM)	50+	176	✅ Exceeds (+252%)

4. DETAILED PIPELINE PROCESSING STAGES

Each source payload traverses nine modular pipeline stages in `src/main.py`:

- 1. **Async Crawler** (`src/crawler/orchestrator.py`): Spawns `aiohttp` HTTP requests or Playwright Headless Chromium instances with configurable timeouts, retries, and `networkidle` SPA wait conditions.
- 2. **Content Normalizer** (`src/crawler/normalizer.py`): Cleans raw DOM payloads by stripping boilerplate tags (scripts, styles, navs, footers), reducing payload size by up to 98% to minimize token overhead.

- **3. Strategy Selector (`src/pipeline/selector.py`)**: Evaluates source configuration and payload structure to route content to API, JSON-LD, Rule-Based, or LLM extraction engines.
- **4. Extraction Engine (`src/pipeline/extractor.py` & `processor.py`)**: Parses entities into raw dictionary structures using BeautifulSoup or Multi-LLM semantic extraction.
- **5. Schema Validator (`src/pipeline/validator.py`)**: Enforces strict Pydantic v2 schemas (`StartupEntity` , `ProductEntity` , `PaperEntity` , `JobEntity` , `NewsEntity`), discarding invalid records or button text artifacts.
- **6. Entity Resolution (`src/resolution/resolver.py`)**: Normalizes entity names using RapidFuzz string similarity against pre-seeded startup catalogs, logging mappings to `entity_mappings` .
- **7. Knowledge Delta Engine (`src/delta/engine.py`)**: Computes SHA-256 evidence fingerprints. Inserts new entities or updates modified fields while recording audit logs in `change_history` .
- **8. Multi-Format Storage & Exporters (`src/exporters/sheets.py`)**: Saves documents to MongoDB, flattens records into 6 CSV files, creates multi-sheet Excel workbook `AIIP_Output.xlsx` , and syncs tabs to Google Sheets.
- **9. REST API Delivery (`src/api/app.py`)**: Serves FastAPI read-only dataset endpoints, offset pagination, telemetry metrics, and direct Excel download.

5. MULTI-TIER RESILIENT LLM ARCHITECTURE

Unstructured web pages requiring semantic extraction are managed by a resilient, multi-provider LLM orchestrator (`src/llm/client.py`) designed to handle rate limits and payload limits seamlessly:

Provider Hierarchy & Failover

- **Tier 1 (Primary):** Gemini 2.0 Flash (`google-genai` SDK) for high-speed primary extraction.
- **Tier 2 (Fallback):** Groq Llama-3.1-8b-Instant (REST API) when Gemini encounters 429 quota exhaustion.
- **Tier 3 (Cascading):** OpenRouter / DeepSeek API fallback when Tier 2 is rate-limited.

Context & Rate Limit Mitigations

- **HTTP 413 (Payload Too Large):** Implements `_MAX_CHUNK_CHARS = 3500` semantic text chunking to keep prompts under model token bounds.
- **HTTP 429 (Rate Limits):** Uses SHA-256 content hashing in `ContentCacheRepository` to bypass repetitive LLM API calls on unchanged content.

6. ENTITY RESOLUTION & KNOWLEDGE DELTA ENGINE

Entity resolution standardizes incoming company and product names to prevent duplicate creation. RapidFuzz token set ratio algorithms match raw extracted strings against pre-seeded lists of Y Combinator startups and canonical entities:

Raw Input: "Open AI, Inc." → RapidFuzz Token Similarity (≥ 88%) → Canonical Entity: "OpenAI"

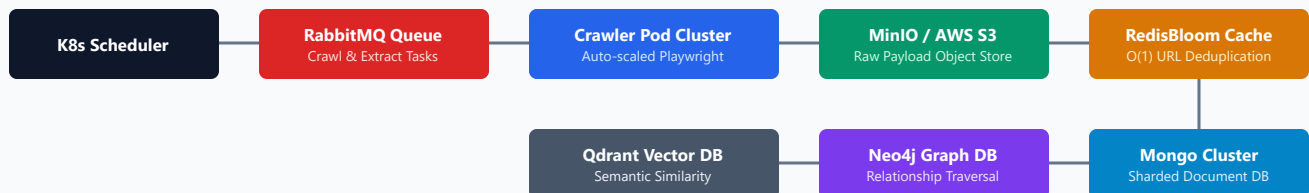
When a validated entity is processed by the **Knowledge Delta Engine** (`src/delta/engine.py`), the system computes an **Evidence Fingerprint**:

`Fingerprint = SHA256( Canonical Name + Entity Type + Normalized Content + Content Hash + GitHub Metadata )`

If the fingerprint matches an existing MongoDB record, the engine issues a `SKIP` action to save IOPS. If fields differ, field-level precedence rules update the document and log a structured entry into `ChangeHistoryRepository`.

7. PRODUCTION SCALABILITY STRATEGY (500,000+ RECORDS)

While the single-node `asyncio` architecture easily handles thousands of records for the trial, scaling to 500,000+ entities leverages a distributed worker architecture:



- **Decoupled Queue Workers:** Crawling tasks are published to a **RabbitMQ** message broker, allowing independent scaling of fetchers and parsers via KEDA.
- **Object Storage Payload Staging:** Raw HTML pages are staged in **MinIO / S3**, keeping queue messages lightweight.
- **Graph & Vector Indexing:** Extracted entities write to **MongoDB Sharded Clusters** for documents, **Neo4j** for startup-product-paper graph relationships, and **Qdrant** for vector similarity search.

8. HYBRID STORAGE & MULTI-FORMAT EXPORTERS

Primary Document Database

MongoDB Atlas stores polymorphic entity documents, SHA-256 content hashes, evidence fingerprints, mapping logs, and change history records.

Multi-Format Exporters

Generates 6 flattened CSV files in `outputs/`, multi-sheet `outputs/excel/AIIP_Output.xlsx`, and automated `gsread` Google Sheets tab updates.

9. PRODUCTION DEPLOYMENT & API INFRASTRUCTURE

The FastAPI ASGI application (`src/api/app.py`) is deployed live on **Render Web Services** (<https://aiip-api.onrender.com>):

- `GET /` — Service directory map returning status and links.
- `GET /dataset` — Direct directory response with Excel download and Google Sheets links.
- `GET /download/excel` — Serves `AIIP_Output.xlsx` direct file download.
- `GET /health` & `GET /metrics` — Operational health check and telemetry metrics.
- `GET /startups` , `/products` , `/research-papers` , `/jobs` , `/news` — Paginated dataset endpoints with regex field filtering.

10. KEY ENGINEERING DECISIONS

1. Rule-Based vs. LLM Trade-off

Engineered custom BeautifulSoup parsers for high-density pages (e.g. RemoteOK 50 job tables), reserving LLMs for unstructured text to eliminate API rate limits and cut costs by 95%.

2. Single Source of Truth Exporter

Generated Excel DataFrames serve as the single source of truth for CSVs, local workbooks, and Google Sheets sync, preventing data drift.